cadence

Cadence USB Driver User Guide

Product Version 1.0.16

January 2018

© 1996-2018 Cadence Design Systems, Inc. All rights reserved.

Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

Trademarks: Trademarks and service marks of Cadence Design Systems, Inc. contained in this document are attributed to Cadence with the appropriate symbol. For queries regarding Cadence's trademarks, contact the corporate legal department at the address shown above or call 800.862.4522. All other trademarks are the property of their respective holders.

Restricted Permission: This document is protected by copyright law and international treaties and contains trade secrets and proprietary information owned by Cadence. Unauthorized reproduction or distribution of this document, or any portion of it, may result in civil and criminal penalties. Except as specified in this permission statement, this document may not be copied, reproduced, modified, published, uploaded, posted, transmitted, or distributed in any way, without prior written permission from Cadence. This document contains the proprietary and confidential information of Cadence or its licensors, and is supplied subject to, and may be used only in accordance with, a written agreement between Cadence and its customer.

Unless otherwise agreed to by Cadence in writing, this statement grants Cadence customers permission to print one (1) hard copy of this document subject to the following conditions:

1. This document may not be modified in any way.

2. Any authorized copy of this document or portion thereof must include all original copyright, trademark, and other proprietary notices and this permission statement.

3. The information contained in this document cannot be used in the development of like products or software, whether for internal or external use, and shall not be used for the benefit of any other party, whether or not for consideration.

Disclaimer: INFORMATION IN THIS DOCUMENT IS SUBJECT TO CHANGE WITHOUT NOTICE AND DOES NOT REPRESENT A COMMITMENT ON THE PART OF CADENCE. EXCEPT AS MAY BE EXPLICITLY SET FORTH IN A WRITTEN AGREEMENT BETWEEN CADENCE AND ITS CUSTOMER, CADENCE DOES NOT MAKE, AND EXPRESSLY DISCLAIMS, ANY REPRESENTATIONS OR WARRANTIES AS TO THE COMPLETENESS, ACCURACY OR USEFULNESS OF THE INFORMATION CONTAINED IN THIS DOCUMENT. CADENCE DOES NOT WARRANT THAT USE OF SUCH INFORMATION WILL NOT INFRINGE ANY THIRD PARTY RIGHTS, AND CADENCE DISCLAIMS ALL IMPLIED WARRANTIES, INCLUDING MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. CADENCE DOES NOT ASSUME ANY LIABILITY FOR DAMAGES OR COSTS OF ANY KIND THAT MAY RESULT FROM USE OF SUCH INFORMATION. CADENCE CUSTOMER HAS COMPLETE CONTROL AND FINAL DECISION-MAKING AUTHORITY OVER ALL ASPECTS OF THE DEVELOPMENT, MANUFACTURE, SALE AND USE OF CUSTOMER'S PRODUCT, INCLUDING, BUT NOT LIMITED TO, ALL DECISIONS WITH REGARD TO DESIGN, PRODUCTION, TESTING, ASSEMBLY, QUALIFICATION, CERTIFICATION, INTEGRATION OF CADENCE PRODUCTS, INSTRUCTIONS FOR USE, LABELING AND DISTRIBUTION, AND CADENCE EXPRESSLY DISAVOWS ANY RESPONSIBILITY WITH REGARD TO ANY SUCH DECISIONS REGARDING CUSTOMER'S PRODUCT.

Restricted Rights: Use, duplication, or disclosure by the Government is subject to restrictions as set forth in FAR52.227- 14 and DFAR252.227-7013 et seq. or its successor.

Cadence USB Driver User Guide: User Guide

Cadence Design Systems

Firmware Identifier:ss.soma-2014-01-13-1a7b081cab45803ec826119632f75c01

Hardware Identifier:e83addbfb7700ee2adf63bc3ef00ef09

Table of Contents

1. Acronyms 1
2. Document Purpose 2
3. Description 3
    3.1. Core Driver Features 3
    3.2. Naming conventions 4
    3.3. System Usage 4
4. Core Driver Usage Model 5
    4.1. Description 5
    4.2. Probing the Hardware 5
    4.3. Initialisation 6
    4.4. Event Handling 8
    4.5. Enumeration 8
    4.6. Data transfer operation 8
    4.7. Quitting Driver 9
    4.8. Core Driver Interrupt/Event Types 9
5. Dynamic Data Structures 10
    5.1. Introduction 10
    5.2. CUSBD_EpConfig 10
    5.3. CUSBD_Config 10
    5.4. CUSBD_SysReq 11
    5.5. CUSBD_ListHead 11
    5.6. CUSBD_SgList 11
    5.7. CUSBD_Req 12
    5.8. CUSBD_Ep 12
    5.9. CUSBD_Dev 13
    5.10. CUSBD_epOps 14
    5.11. CUSBD_Callbacks 14
    5.12. CUSBD_DMAInterfaceWidth 15
    5.13. CUSBD_PreciseBurstLength 15
    5.14. CUSBD_reqComplete 16
    5.15. CUSBD_epEnable 16
    5.16. CUSBD_epDisable 16
    5.17. CUSBD_epSetHalt 17
    5.18. CUSBD_epSetWedge 17
    5.19. CUSBD_epFifoStatus 17
    5.20. CUSBD_epFifoFlush 18
    5.21. CUSBD_reqQueue 18
    5.22. CUSBD_reqDequeue 18
    5.23. CUSBD_reqAlloc 19
    5.24. CUSBD_reqFree 19
    5.25. CUSBD_CbDisconnect 19
    5.26. CUSBD_CbConnect 19
    5.27. CUSBD_CbSetup 20
    5.28. CUSBD_CdSuspend 20
    5.29. CUSBD_CbResume 20
    5.30. CUSBD_CbUsbRequestMemAlloc 21
    5.31. CUSBD_CbUsbRequestMemFree 21
    5.32. CUSBD_CbbusInterval 21
6. Driver API Object 22

6.1. Introduction 22
6.2. CUSBD_GetInstance 22
6.3. probe 22
6.4. init 23
6.5. destroy 23
6.6. start 24
6.7. stop 24
6.8. isr 25
6.9. epEnable 25
6.10. epDisable 26
6.11. epSetHalt 26
6.12. epSetWedge 27
6.13. epFifoStatus 28
6.14. epFifoFlush 28
6.15. reqQueue 29
6.16. reqDequeue 29
6.17. reqAlloc 30
6.18. reqFree 31
6.19. getDevInstance 31
6.20. dGetFrame 32
6.21. dWakeUp 32
6.22. dSetSelfpowered 33
6.23. dClearSelfpowered 33
6.24. dVbusSession 34
6.25. dVbusDraw 34
6.26. dPullUp 35
6.27. dExecDebug 36
6.28. dGetConfigParams 36

List of Figures

3.1. Calling callback scheme 3
3.2. CUSBD objects 4

List of Tables

4.1. Callback Functions and Events 9

Chapter 1. Acronyms

API	Application Programming Interface.
Core Driver	Cadence Firmware component that provides IP programming abstraction.
CPS	Cadence Platform Services. A set of basic platform specific access functions acting as hardware abstraction layer for the core driver to operate.
USB	Universal Serial Bus
CUSBD	Cadence USB Device driver

Chapter 2. Document Purpose

This document is intended to serve as an overview and a reference for customers looking to use the Cadence USB Firmware Core Driver for the Cadence USB Device IP Core.

Chapter 3. Description

The FW provided by Cadence is of two types:

1. Production Code: Code built to production standards. i.e. it is fully verified with positive and negative testing.
2. Reference Code: This is example code for the platform specific portions of the customers' code base. Reference code is only provided as an example to help the customer with his environment. Ideally code snippets from this will be looked at, and the customer will create his own versions of the reference code that is specific to his environment.

3.1. Core Driver Features

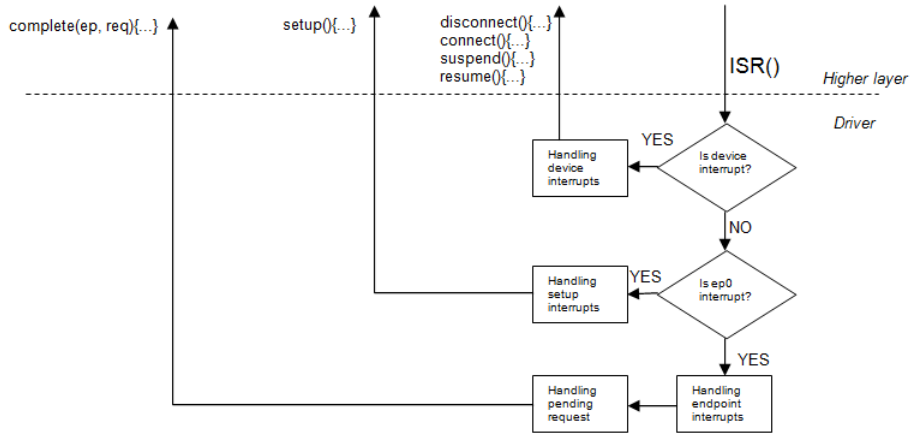
Supports all features of the Core:

1. Super Speed, High Speed and Full Speed function interface
2. Supports up to 15 IN and 15 OUT configurable/programable endpoints
3. Endpoint supports bulk, interrupt, isochronous type transfer
4. Support for streams
5. Control transfers supported by endpoint 0
6. Full power management capabilities(U1, U2 and U3) with LFPS support
7. Certified by USB-IF for compliance with the latest revision of USB 3.0 specification

The *Init* function configures a number of parameters on creation of the driver instance, e.g. register base address, DMA interface width, the callback functions

Provide event polling API for upper layer ISR - the driver's isr() function should be called in response to a hardware interrupt, and will notify of events by means of client-defined callback functions, e.g. setup packet received, disconnection from host, transfer complete, etc.

Figure 3.1. Calling callback scheme



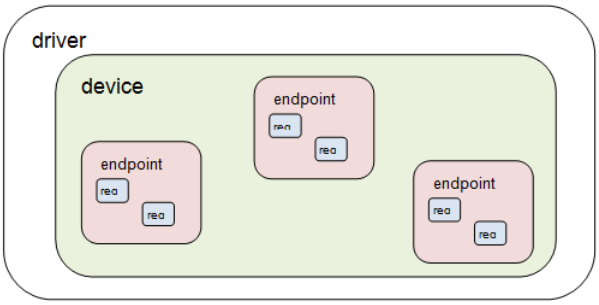
Support multi-processor scenario

- To allow concurrent processing, the callbacks should be used to queue events for processing after the end of the isr() call, since this will have to lock common driver resources during its execution

Driver functions

Driver is completely written in C but its structure resembles object structure:

Figure 3.2. CUSBD objects



There are three groups of functions: related to device, endpoint and request

Detailed data structures and APIs are listed in the later chapters of the document.

3.2. Naming conventions

The CUSBD Core Driver API will use the prefix CUSBD_ for public top-level names of functions and data objects/ definitions.

3.3. System Usage

The CUSBD Core Driver is supplied with linux driver reference code which illustrates the use of the core driver in the wider system context. For more details on this see the linux driver Readme file.

Chapter 4. Core Driver Usage Model

4.1. Description

In order to support multiple instances and provide the minimum name space pollution, the Core Driver is implemented as a structure that contains a function pointer table.

A public header is provided to define the API object structure and the data structures used. The file is contained in the include directory of the distribution as *cusbd_if.h*.

The API requires a private data pointer that provides the driver with instance information. It is critical that the upper software layer abides by the programming model.

Every API call apart from *probe* requires the private data address as the first parameter, to identify the driver instance and provide access to the internal state of that instance.

There is no resource protection mechanism provided in the CUSBD Core Driver. It is expected that the higher level software stack will be responsible in protecting calls to the core driver where appropriate, for example when queuing a multi-buffer frame for transmission. In critical areas such as the calling of the *isr* function, usage is expected to be single threaded. Multi-threaded processing of a received frame is possible, provided that each priority queue handles its own buffer freeing and recycling after any callback function has returned.

4.2. Probing the Hardware

The client code must obtain the object pointer to the core driver. It can be retrieved by issuing the *CUSBD_GetInstance()* function. This provides access to all of the API functions, but initially only *probe* may be called.

Before initialising the core driver, the client must first invoke the *probe* function. The *probe* function returns the dynamic memory requirements for the driver, depending on the configuration specified. The configuration information is passed to *probe* in an *CUSBD_Config* struct, although at this stage only those fields which affect the requirements need to be set, as indicated in the struct description. The client may repeat *probe* calls to customise configuration depending on available system resources.

The client code then must allocate the memory to the sizes returned by the probe function. One of these allocations is for the driver's device objects represented as privateData. Device objects contains number of endpoint objects given in configuration. Each endpoint must have memory space allocated by client

```
...
CUSBD_Config config = {
    .regBase = USBR_BASE, // address where USB core is mapped
    .epIN =
    {
        {.bufferingValue = 1}, // ep1in
        {.bufferingValue = 0}, // ep2in
        {.bufferingValue = 0}, // ep3in
        {.bufferingValue = 0}, // ep4in
        {.bufferingValue = 0}, // ep5in
        {.bufferingValue = 0}, // ep6in
        {.bufferingValue = 0}, // ep7in
        {.bufferingValue = 0}, // ep8in
        {.bufferingValue = 0}, // ep9in
        {.bufferingValue = 0}, // ep10in
        {.bufferingValue = 0}, // ep11in
```

```
        {.bufferingValue = 0}, // ep12in
        {.bufferingValue = 0}, // ep13in
        {.bufferingValue = 0}, // ep14in
        {.bufferingValue = 0} // ep15in
    },
    .epOUT =
    {
        {.bufferingValue = 0}, //ep1out
        {.bufferingValue = 0}, //ep2out
        {.bufferingValue = 0}, //ep3out
        {.bufferingValue = 0}, //ep4out
        {.bufferingValue = 0}, //ep5out
        {.bufferingValue = 0}, //ep6out
        {.bufferingValue = 0}, //ep7out
        {.bufferingValue = 0}, //ep8out
        {.bufferingValue = 0}, //ep9out
        {.bufferingValue = 0}, //ep10out
        {.bufferingValue = 0}, //ep11out
        {.bufferingValue = 0}, //ep12out
        {.bufferingValue = 0}, //ep13out
        {.bufferingValue = 0}, //ep14out
        {.bufferingValue = 0} //ep15out
    },
    .dmultEnabled = 0, // set to 1 if scatter/gather DMA available
    .dmaInterfaceWidth = DMA_64_WIDTH,
    .preciseBurstLength = PRECISE_BURST_0
};


static CUSBD_SysReq sysReq;
uint32_t res; // keeps result of operation on driver
drv = CUSBD_GetInstance(); // get driver handle

res = drv->probe(&config, &sysReq);
if (res != 0) goto error;
printf("Device memory requirement: %d bytes\n", sysReq.privDataSize);
printf("Single endpoint memory requirement: %d bytes\n", sysReq.epObjSize);
...
```

4.3. Initialisation

After this stage, the client must reserve the necessary memory areas for the driver's private data. The CUSBD_Config struct should be filled out with endpoint configuration vectors for IN and OUT direction, and values for all the other initial configuration fields.

The driver can now be initialised with an *init* call, which will:

- initialise the driver instance

- configure the hardware as specified in the *config* parameter fields

- initializes selected IN and OUT endpoints

- set initial USB device state to default state

e.g. *init* example, following on from *probe* example above:

```
...

static CUSBD_OBJ * drv;     // driver pointer
static void * pD;           // private data pointer

static void connect(void *pD) {..}

static void disconnect(void *pD) {...}

static void resume(void *pD) {...}

static void setup(void *pD, CH9_UsbSetup *ctrl) {...}

static void suspend(void *pD) {...}

static void * requestMemAlloc(void * pD, uint32_t requireSize)  {
    return malloc(requireSize);
}

static void requestMemFree(void * pD, void* usbRequest)  {
    if (usbRequest)
        free(usbRequest);
}

CUSBD_Callbacks callback = {
    .connect = connect,
    .disconnect = disconnect,
    .resume = resume,
    .setup = setup,
    .suspend = suspend,
    .usbRequestMemAlloc = requestMemAlloc,
    .usbRequestMemFree = requestMemFree
};

int appStart() {

    uint32_t res;              // keeps result of operation on driver
    CUSBD_SysReq sysReq;       // keeps driver memory requirements
    drv = CUSBD_GetInstance(); // get driver handle

    res = drv->probe(&config, &sysReq);    // probe driver
    if (res != 0) goto error;
    vDbgMsg(DBG_USB_CTRL_MSG, 1, "Device memory requirement: %ld bytes\n", sysReq.privDataSize);

    // allocate memory for device object
    pD = (uint8_t*) malloc((size_t) (sysReq.privDataSize));
    if(!pD) goto error;
    // allocate memory for Transfer Descriptor Blocks
    config.trbAddr = malloc(sysReq.trbMemSize);
    if(!config.trbAddr)
        goto error;

    // In Bare Metal physical and virtual space are the same space
    config.trbDmaAddr = (uintptr_t)config.trbAddr;
    // initialize driver
    res = drv->init(pD, &config, &callback);
    if (res != 0) goto error;
```

At this point, any further setup not provided by CUSBD_Config should be done.

When ready, *start* should be called to enable the interrupts.

4.4. Event Handling

The Core Driver notifies upper layer software of new events and state changes by means of the *isr* API function. The *isr* function may be invoked by upper layer software as part of the interrupt processing being handled by the upper layer software. Alternatively, if interrupt-driven operation is not desired, *isr* can be called by frequent polling. In the latter case, the CUSBD interrupt should not lead to a read of the interrupt status register by the client software, as this must be interpreted and cleared by the Core Driver's *isr* function.

The invocation of the *isr* function of the Core Driver will result in a callback to the function registered as part of the *init* process IF there is an enabled event or error condition that is encountered by the Core driver. It is the responsibility of the upper layer software to deal with any errors that are reported by the Core driver.

Actions performed by the upper layer during a callback function should be kept to a minimum, as the function must return before further *isr* event-checking can proceed. Data passed by pointer in callbacks (e.g. setup callback function transfers 8-bytes length USB protocol setup packet in ctrl parameter.

4.5. Enumeration

Descriptors used during enumeration must be implemented on client side. Descriptors differs according to implemented USB device application and should be written in forms of structures defined in cusb_ch9.h. cusb_ch9.h contains definition and types used only in standard framework. Class or vendor specific extension must be added individually by driver's client. Descriptors should be defined as global static structure variables and should be accessible whenever driver's runtime. Example device descriptor for Super Speed mode is shown below:

```
// device descriptor for SuperSpeed mode
static CDN_UusbDeviceDescriptor devSsDesc = {
    .bLength = CH9_USB_DS_DEVICE,
    .bDescriptorType = CH9_USB_DT_DEVICE,
    .bcdUSB = cpu_to_le16(BCD_USB_SS),
    .bDeviceClass = 0,
    .bDeviceSubClass = 0,
    .bDeviceProtocol = 0,
    .bMaxPacketSize0 = 9,
    .idVendor = cpu_to_le16(ID_VENDOR),
    .idProduct = cpu_to_le16(ID_PRODUCT),
    .bcdDevice = cpu_to_le16(BCD_DEVICE_SS),
    .iManufacturer = 1,
    .iProduct = 2,
    .iSerialNumber = 3,
    .bNumConfigurations = 1
};
```

Handling host's setup requests

Host's setup request received by USB device core is dispatched by driver to client-defined *setup* callback. In most cases handling of setup requests rely on sending selected descriptors to host. Transfering data to or from host is described in next section of this document. Class specific request should be handled according to class specification. Not recognized request should be completed by returning ENOTSUP error. For succesfully completed requests setup callback function should return 0. Remember that *ctrl* parameter is given in little endian

4.6. Data transfer operation

Data transfer is possible only on endpoint object. Acording to USB specification, default endpoint must be enabled at once on start-up.

Default endpoint

Another name for default endpoint is *endpoint zero*. This endpoint is bi-directional and configured to work on driver start. Endpoint zero doesn't queue requests. It means that host requests must be handled sequentially - next request can start only after completion of previous transfer. Default endpoint is accessible as field of CUSBD object. To invoke selected operation on endpoint use: dev->ep0->ops->... where dev is a CUSBD (usb device) object.

Bulk, interrupt endpoints

According to USB specification all endpoints except default endpoint should be configured until SET_CONFIGURATION(>0) setup request. To enable endpoint use epEnable function. To start transfer on endpoint use reqQueue function. Driver queues transfer requests for each endpoint separately. epEnable function is no-blocking type. It means that function allways returns immediatelly regardless data state. The result of transfer operation is returned in *complete* callback function. Callback passes as parameters endpoint and request for which the transfer has completed. Reading status field of request gives information about completed transfer status, 0 - transfer completed successfully, ECANCELED - transfer was canceled for some reasons (e.g. disconnect). Complete transfer callback requests are called in order the corresponding transfer requests were queued.

Isochronous endpoints

Endpoint configuration and data transfer is realised in the same way as in bulk or interrupt endpoint. The only one relevant difference exists - isochronous endpoint doesn't generates transfer completion. Even if defined, complete callback will not be called on data transfer.

4.7. Quitting Driver

The *stop* function can be called to end the driver's active operation.

At the end of the driver instance's life, *destroy* should be called to terminate use.

This stops any ongoing activity, disables the hardware and de-initialises the driver. The driver instance cannot be used after this call.

4.8. Core Driver Interrupt/Event Types

Table 4.1. Callback Functions and Events

Callback Function	Description
disconnect	Device is disconnected from host
connect	Device connected to host
suspend	Device goes to suspend state
resume	Resume device to normal operation
setup	Setup packet has been received
complete	Request transfer complete

Chapter 5. Dynamic Data Structures

5.1. Introduction

This section defines the data structures used by the driver to provide hardware information, modification and dynamic operation of the driver.

These data structures are defined in the header file of the core driver and utilized by the API.

5.2. CUSBD_EpConfig

Type: struct

Description

Configuration structure for single endpoint.

Single endpoint configuration structure object is a part of

Fields

`bufferingValue` buffering value It is a number of hardware (on-chip) memory buffer used for queuing USB packets.

Setting bufferingValue to 0 means that endpoint is disabled Setting bufferingValue to 1 it sets number of on-chip buffer to 1 and so on.

`supportStream` Bulk endpoint support streams, used only in Super Speed mode.

5.3. CUSBD_Config

Type: struct

Description

Configuration of device.

Object of this type is used for probe and init functions when checking memory requirements of device object and for hardware controller configuration. Size of device object in bytes is returned on probe function call in

Fields

`regBase` base address of device controller registers

`epIN` table of endpoint configuration objects for IN direction

`epOUT` table of endpoint configuration objects for OUT direction

`dmultEnabled` hardware feature enabling DMA operation in scatter/gather mode 1 - enabled, 0 disabled

`dmaInterfaceWidth` DMA interface width, available values: DMA_32_WIDTH, DMA_64_WIDTH.

`preciseBurstLength` Set configurable burst length in DMA, available values: PRECISE_BURST_0, PRECISE_BURST_1, PRECISE_BURST_2, PRECISE_BURST_4,

PRECISE_BURST_8, PRECISE_BURST_16, PRECISE_BURST_32, PRECISE_BURST_64, PRECISE_BURST_128.

`trbAddr` Pointer to virtual memory for TRBs pool.

`trbDmaAddr` Address of physical memory for TRBs pool.

5.4. CUSBD_SysReq

Type: struct

Description

System requirements returned by probe.

Fields

`privDataSize` size of memory required for driver's private data

`epObjSize` size of memory required for endpoint object

`trbMemSize` size of memory required for USB Transfer Request Descriptors.

This memory will be used by DMA controller, so it should be suitable for DMA operation.

5.5. CUSBD_ListHead

Type: struct

Description

structure helpful when object are organized in linked list Device contains list of endpoints, in turn: endpoint contains request list

Fields

`next` pointer to next element in list

`prev` pointer to previous element in list

5.6. CUSBD_SgList

Type: struct

Description

structure contains information about memory slices for scatter/gather transfer

Fields

`link` link to the next elemen

`offset`	offset within memory page
`length`	data length of this transfer slice
`dmaAddress`	physical address of buffer to read/write

5.7. CUSBD_Req

Type: struct

Description

Transfer request structure.

I/O transfers of higher layers are realized with the help of

Fields

`list`	request can be organized in linked list
`buf`	buffer with data to transfer - virtual address
`length`	This transfer data length.
`dma`	physical address of buf buffer
`sg`	scatter/gather list
`numOfSgs`	number of scatter/gather entries
`numMappedSgs`	number of scatter/gather entries mapped for hardware controller
`streamId`	stream id - used only in Super Speed mode
`noInterrupt`	Setting this flag to 1 disables generating of transfer completion interrupt.
`zero`	Setting this flag to 1 causes adding zero length data packet at the end of data transfer when data transfer length is a multiple of MaxPacketSize for endpoint.
`shortNotOk`	Setting this flag causes all reading transfers with short packet as erroneous.
`complete`	Callback function called on the transfer completion event.
`context`	general-use data hook driver doesn't make use of this field. May be dsed in complete callback function
`status`	keeps actual status of request
`actual`	number of actually processed bytes

5.8. CUSBD_Ep

Type: struct

Description

Structure represents device USB endpoint.

Fields

`epList`	enables organization of endpoints in linked list
`driverData`	not used by CUSBD, used by higher layers
`name`	keeps name of endpoint
`ops`	endpoint driver, contains methods for all operation required on endpoint
`maxPacket`	Maximal packet size for endpoint.
`maxStreams`	Maximal number of streams used by endpoint, useful only in SS mode.
`mult`	packet multiplication, it enables to multiplicate number of packet per service interval, useful in HS and SS mode only for isochronous endpoint, mult value range from 0 to 2 - it multiplicate packets by 1 to 3
`maxburst`	Maximal number of packets in burst transfer, useful only in SS mode maxburst value range from 0 to 15 giving number of packets 1 to 16.
`address`	Endpoint address, the oldest bit refers to endpoint direction. For example: address = 0x82 refers to endpoint 2 IN address = 0x03 refers to endpoint 3 OUT
`desc`	pointer to user defined endpoint descriptor, CH9_UsbEndpointDescriptor is defined in cusb_ch9.h
`compDesc`	pointer to user defined endpoint companion descriptor, useful only in SS mode, CH9_UsbSSEndpointCompanionDescriptor is defined in cusb_ch9.h

5.9. CUSBD_Dev

Type: struct

Description

Structure represents USB device.

Fields

`epList`	allows for manipulation on endpoints organized as linked list
`ep0`	pointer to bidirectional default endpoint, this endpoint should be always available after driver start
`speed`	speed value in which device actually works
`maxSpeed`	maximal speed the device is able to work

`state`	actual state in which device actually works
`sgSupported`	Flags informs if device is scatter/gather transfer capable.
`isOtg`	Informs if device is OTG type.
`isAPeripheral`	Informs if device is A peripheral type.
`bHnpEnable`	Informs if device is HNP capable.
`aHnpSupport`	informs if OTG device in host mode supports HNP on this port
`name`	Full name of USB device controller.

5.10. CUSBD_epOps

Type: struct

Description

Set of functions allowing to operate on endpoints.

Fields

`epEnable`

`epDisable`

`epSetHalt`

`epSetWedge`

`epFifoStatus`

`epFifoFlush`

`reqQueue`

`reqDequeue`

`reqAlloc`

`reqFree`

5.11. CUSBD_Callbacks

Type: struct

Description

struct containing function pointers for event notification callbacks issued by isr().

Each call passes the driver's privateData pointer for instance identification if necessary, and may also pass data related to the event.

Fields

`disconnect`

`connect`

`setup`

`suspend`

`resume`

`usbRequestMemAlloc`

`usbRequestMemFree`

`busInterval`

5.12. CUSBD_DMAInterfaceWidth

Type: enum

Description

DMA BUS width available values.

Values

`CUSBD_DMA_32_WIDTH` Value: 4

`CUSBD_DMA_64_WIDTH` Value: 8

5.13. CUSBD_PreciseBurstLength

Type: enum

Description

Precise burst available values.

Values

`CUSBD_PRECISE_BURST_0` Value: 0x00000000L

`CUSBD_PRECISE_BURST_1` Value: 0x01000000L

`CUSBD_PRECISE_BURST_2` Value: 0x02000000L

`CUSBD_PRECISE_BURST_4` Value: 0x04000000L

`CUSBD_PRECISE_BURST_8` Value: 0x08000000L

`CUSBD_PRECISE_BURST_16` Value: 0x10000000L

`CUSBD_PRECISE_BURST_32` Value: 0x20000000L

`CUSBD_PRECISE_BURST_64` Value: 0x40000000L

`CUSBD_PRECISE_BURST_128` Value: 0x80000000L

5.14. CUSBD_reqComplete

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

```
void(* CUSBD_reqComplete )(struct CUSBD_Ep *ep, struct CUSBD_Req *req)
```

Description

Reports request complete callback.

Params: ep - endpoint associated with the request req - request which has been completed Note that request received in completion routine may be used again

5.15. CUSBD_epEnable

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

```
uint32_t(* CUSBD_epEnable )(void *pD, struct CUSBD_Ep *ep, const struct CH9_UsbEndpointDescriptor *desc)
```

Description

Enable Endpoint.

This function should be called within SET_CONFIGURATION(configNum > 0) request context. It configures required hardware controller endpoint with parameters given in desc.

5.16. CUSBD_epDisable

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

```
uint32_t(* CUSBD_epDisable )(void *pD, struct CUSBD_Ep *ep)
```

Description

Disable Endpoint.

Functions unconfigures hardware endpoint. Endpoint will not respond to any host packets. This function should be called within SET_CONFIGURATION(configNum = 0) request context or on disconnect event. All queued requests on endpoint are completed with ECANCELED status.

5.17. CUSBD_epSetHalt

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

```
uint32_t(* CUSBD_epSetHalt )(void *pD, struct CUSBD_Ep *ep, uint8_t value)
```

Description

Set halt or clear halt state on endpoint.

When setting halt, device will respond with STALL packet to each host packet. When clearing halt, endpoint returns to normal operating.

5.18. CUSBD_epSetWedge

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

```
uint32_t(* CUSBD_epSetWedge )(void *pD, struct CUSBD_Ep *ep)
```

Description

Set halt on endpoint.

Function sets endpoint to permanent halt state. State can not be changed even with epSetHalt(pD, ep, 0) function. Endpoint returns automatically to its normal state on disconnect event.

5.19. CUSBD_epFifoStatus

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

```
uint32_t(* CUSBD_epFifoStatus )(void *pD, struct CUSBD_Ep *ep)
```

Description

Returns number of bytes in hardware endpoint FIFO.

Function useful in application where exact number of data bytes is required. In some situation software higher layers must be aware of number of data bytes issued but not transfered by hardware because of aborted transfers, for example on disconnect event.

5.20. CUSBD_epFifoFlush

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

```
void(* CUSBD_epFifoFlush )(void *pD, struct CUSBD_Ep *ep)
```

Description

Flush hardware endpoint FIFO.

5.21. CUSBD_reqQueue

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

```
uint32_t(* CUSBD_reqQueue )(void *pD, struct CUSBD_Ep *ep, struct CUSBD_Req *req)
```

Description

Submits IN/OUT transfer request to an endpoint.

5.22. CUSBD_reqDequeue

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

```
uint32_t(* CUSBD_reqDequeue )(void *pD, struct CUSBD_Ep *ep, struct CUSBD_Req *req)
```

Description

Dequeues IN/OUT transfer request from an endpoint.

Function completes all queued request with ECANCELED status.

5.23. CUSBD_reqAlloc

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

```
uint32_t(* CUSBD_reqAlloc )(void *pD, struct CUSBD_Ep *ep, struct CUSBD_Req **req)
```

Description

Allocates transfer request object.

Function returns address to memory where request is casted.

5.24. CUSBD_reqFree

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

```
void(* CUSBD_reqFree )(void *pD, struct CUSBD_Ep *ep, struct CUSBD_Req *req)
```

Description

Frees memory occupied by request.

While operating, when request object will be no longer used, use reqFree function to free system memory

5.25. CUSBD_CbDisconnect

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

```
void(* CUSBD_CbDisconnect )(void *pD)
```

Description

Callback called on disconnect from host event.

All queued transfer on all active endpoints are completed by driver with ECANCELED status.

5.26. CUSBD_CbConnect

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

```
void(* CUSBD_CbConnect )(void *pD)
```

Description

Callback called on connection to host event.

To check speed at which device has connected to host read speed flag of CUSBD object

5.27. CUSBD_CbSetup

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

```
uint32_t(* CUSBD_CbSetup )(void *pD, struct CH9_UsbSetup *ctrl)
```

Description

Callback called on setup request received event Params: ctrl contains usb setup request in little endian order.

5.28. CUSBD_CdSuspend

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

```
void(* CUSBD_CdSuspend )(void *pD)
```

Description

Called when device controller goes into suspend state.

5.29. CUSBD_CbResume

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

```
void(* CUSBD_CbResume )(void *pD)
```

Description

Called when device returns from suspend state.

5.30. CUSBD_CbUsbRequestMemAlloc

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

```
void *(* CUSBD_CbUsbRequestMemAlloc )(void *pD, uint32_t requiredSize)
```

Description

Called when device needs allocation memory for CUSBD_Req object.

5.31. CUSBD_CbUsbRequestMemFree

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

```
void(* CUSBD_CbUsbRequestMemFree )(void *pD, void *usbRequest)
```

Description

Called when device needs free memory for CUSBD_Req object.

5.32. CUSBD_CbbusInterval

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

```
void(* CUSBD_CbbusInterval )(void *pD)
```

Description

Called on bus interval event only if any isochronous endpoint used.

Chapter 6. Driver API Object

6.1. Introduction

API listing for the driver.

The API is contained in the object as function pointers in the object structure. As the actual functions resides in the Driver Object, the client software must first use the global GetInstance function to obtain the Driver Object Pointer. The actual APIs then can be invoked using obj->(api_name)() syntax. These functions are defined in the header file of the core driver and utilized by the API.

6.2. CUSBD_GetInstance

Function Declaration

```
CUSBD_OBJ* CUSBD_GetInstance();

void
```

Description

In order to access the CUSBD APIs, the upper layer software must call this global function to obtain the pointer to the driver object.

Return Value

CUSBD_OBJ* Driver Object Pointer

CUSBD_OBJ

6.3. probe

Function Declaration

```
uint32_t probe(config, sysReq);

CUSBD_Config* config;
CUSBD_SysReq* sysReq;
```

Parameter List

`sysReq` [out]

returns the size of memory allocations required

`config` [in]

driver/hardware configuration required

Description

Obtain the private memory size required by the driver.

Return Value

0 on success (requirements structure filled)

ENOTSUP if configuration cannot be supported due to driver/hardware constraints

6.4. init

Function Declaration

```
uint32_t init(pD, config, callbacks);

void* pD;
const CUSBD_Config* config;
CUSBD_Callbacks* callbacks;
```

Parameter List

`callbacks` [in]

client-supplied callback functions

`config` [inout]

specifies driver/hardware configuration

`pD` [in]

driver state info specific to this instance

Description

Initialize the driver instance and state, configure the USB device as specified in the 'config' settings, initialize locks used by the driver.

Return Value

0 on success

ENOTSUP if hardware has an inconsistent configuration or doesn't support feature(s) required by 'config' parameters

ENOENT if insufficient locks were available (i.e. something allocated locks between probe and init)

EIO if driver encountered an error accessing hardware

EINVAL if illegal/inconsistent values in 'config'

6.5. destroy

Function Declaration

```
void destroy(pD);
```

```
void* pD;
```

Parameter List

`pD` [in]

driver state info specific to this instance

Description

Destroy the driver (automatically performs a stop).

6.6. start

Function Declaration

```
void start(pD);
```

```
void* pD;
```

Parameter List

`pD` [in]

driver state info specific to this instance

Description

Start the USB driver.

6.7. stop

Function Declaration

```
void stop(pD);
```

```
void* pD;
```

Parameter List

`pD` [in]

driver state info specific to this instance

Description

Stop the driver.

This should disable the hardware, including its interrupt at the source, and make a best-effort to cancel any pending transactions.

6.8. isr

Function Declaration

```
void isr(pD);

void* pD;
```

Parameter List

`pD` [in]

driver state info specific to this instance

Description

Driver ISR.

Platform-specific code is responsible for ensuring this gets called when the corresponding hardware's interrupt is asserted. Registering the ISR should be done after calling init, and before calling start. The driver's ISR will not attempt to lock any locks, but will perform client callbacks. If the client wishes to defer processing to non-interrupt time, it is responsible for doing so.

6.9. epEnable

Function Declaration

```
uint32_t epEnable(pD, ep, desc);

void* pD;
CUSBD_Ep* ep;
const CH9_UsbEndpointDescriptor* desc;
```

Parameter List

`ep` [in]

endpoint being configured

`pD` [in]

driver state info specific to this instance

`desc` [in]

endpoint descriptor

Description

Enable Endpoint.

This function should be called within SET_CONFIGURATION(configNum > 0) request context. It configures required hardware controller endpoint with parameters given in desc.

Return Value

0 on success

EINVAL if pD, ep or desc is NULL or desc is not a endpoint descriptor

6.10. epDisable

Function Declaration

```
uint32_t epDisable(pD, ep);

void* pD;
CUSBD_Ep* ep;
```

Parameter List

`ep` [in]

endpoint being unconfigured

`pD` [in]

driver state info specific to this instance

Description

Disable Endpoint.

Functions unconfigures hardware endpoint. Endpoint will not respond to any host packets. This function should be called within SET_CONFIGURATION(configNum = 0) request context or on disconnect event. All queued requests on endpoint are completed with ECANCELED status.

Return Value

0 on success

EINVAL if pD or ep is NULL

6.11. epSetHalt

Function Declaration

```
uint32_t epSetHalt(pD, ep, value);

void* pD;
CUSBD_Ep* ep;
uint8_t value;
```

Parameter List

`value` [in]

if 1 sets halt, if 0 clears halt on endpoint

`ep` [in]

endpoint being setting or clearing halt state

`pD` [in]

driver state info specific to this instance

Description

Set halt or clear halt state on endpoint.

When setting halt, device will respond with STALL packet to each host packet. When clearing halt, endpoint returns to normal operating.

Return Value

0 on success

EPERM if endpoint is disabled (has not been configured yet)

EINVAL if pD or ep is NULL

6.12. epSetWedge

Function Declaration

```
uint32_t epSetWedge(pD, ep);

void* pD;
CUSBD_Ep* ep;
```

Parameter List

`ep` [in]

endpoint being setting halt state

`pD` [in]

driver state info specific to this instance

Description

Set halt on endpoint.

Function sets endpoint to permanent halt state. State can not be changed even with epSetHalt(pD, ep, 0) function. Endpoint returns automatically to its normal state on disconnect event.

Return Value

0 on success

EPERM if endpoint is disabled (has not been configured yet)

EINVAL if pD or ep is NULL

6.13. epFifoStatus

Function Declaration

```
uint32_t epFifoStatus(pD, ep);

void* pD;
CUSBD_Ep* ep;
```

Parameter List

ep [in]

endpoint which status is to be returned

pD [in]

driver state info specific to this instance

Description

Returns number of bytes in hardware endpoint FIFO.

Function useful in application where exact number of data bytes is required. In some situation software higher layers must be aware of number of data bytes issued but not transfered by hardware because of aborted transfers, for example on disconnect event. *

Return Value

number of bytes in hardware endpoint FIFO

EINVAL if pD or ep is NULL

6.14. epFifoFlush

Function Declaration

```
void epFifoFlush(pD, ep);

void* pD;
CUSBD_Ep* ep;
```

Parameter List

ep [in]

endpoint which FIFO is to be flushed

pD [in]

driver state info specific to this instance

Description

Flush hardware endpoint FIFO.

6.15. reqQueue

Function Declaration

```
uint32_t reqQueue(pD, ep, req);

void* pD;
CUSBD_Ep* ep;
CUSBD_Req* req;
```

Parameter List

`req` [in]

request being submitted

`ep` [in]

endpoint associated with the request

`pD` [in]

driver state info specific to this instance

Description

Submits IN/OUT transfer request to an endpoint.

Return Value

0 on success

EINVAL if pD, ep, or req is NULL

EPERM if endpoint is not enabled

EPROTO only on default endpoint if endpoint is not in data stage phase

6.16. reqDequeue

Function Declaration

```
uint32_t reqDequeue(pD, ep, req);

void* pD;
CUSBD_Ep* ep;
CUSBD_Req* req;
```

Parameter List

`req` [in]

request being dequeued

`ep` [in]

endpoint associated with the request

`pD` [in]

driver state info specific to this instance

Description

Dequeues IN/OUT transfer request from an endpoint.

Function completes all queued request with ECANCELED status.

Return Value

0 on success

EINVAL if pD or ep is NULL

6.17. reqAlloc

Function Declaration

```
uint32_t reqAlloc(pD, ep, *req);

void* pD;
CUSBD_Ep* ep;
CUSBD_Req* *req;
```

Parameter List

`req` [out]

memory space where request will be allocated

`ep` [in]

endpoint for which request is being allocated

`pD` [in]

driver state info specific to this instance

Description

Allocates transfer request object.

Function returns address to memory where request is casted.

Return Value

NULL if no available memory space

request pointer to CUSBD_Req empty object

6.18. reqFree

Function Declaration

```
void reqFree(pD, ep, req);

void* pD;
CUSBD_Ep* ep;
CUSBD_Req* req;
```

Parameter List

`req` [in]

request being freed

`ep` [in]

endpoint associated with the request

`pD` [in]

driver state info specific to this instance

Description

Frees memory occupied by request.

While operating, when request object will be no longer used, use reqFree function to free system memory *

6.19. getDevInstance

Function Declaration

```
void getDevInstance(pD, *dev);

void* pD;
CUSBD_Dev* *dev;
```

Parameter List

`dev` [out]

returns pointer to CUSBD instance

`pD` [in]

driver state info specific to this instance

Description

Returns pointer to CUSBD object.

CUSBD object is a logical representation of USB device. CUSBD contains endpoint collection accessed through epList field. Endpoints collection is organized as double linked list.

6.20. dGetFrame

Function Declaration

```
uint32_t dGetFrame(pD, numOfFrame);

void* pD;
uint32_t* numOfFrame;
```

Parameter List

`numOfFrame` [out]

returns number of USB frame

`pD` [in]

driver state info specific to this instance

Description

Returns number of frame.

Some controllers have counter of SOF packets or ITP packets in the Super Speed case. Function returns value of this counter. This counter can be used for time measurement. Single FS frame is 1 ms measured, for HS and SS is 125us.

Return Value

0 on success

EINVAL if pD or numOfFrame is NULL

EOPNOTSUPP if feature is not supported

6.21. dWakeUp

Function Declaration

```
uint32_t dWakeUp(pD);

void* pD;
```

Parameter List

`pD` [in]

driver state info specific to this instance

Description

Wakes up host connected to this device.

Return Value

0 on success

EINVAL if pD is NULL

EOPNOTSUPP if feature is not supported

6.22. dSetSelfpowered

Function Declaration

```
uint32_t dSetSelfpowered(pD);

void* pD;
```

Parameter List

`pD` [in]

driver state info specific to this instance

Description

Sets the device self powered feature.

Return Value

0 on success

EINVAL if pD is NULL

EOPNOTSUPP if feature is not supported

6.23. dClearSelfpowered

Function Declaration

```
uint32_t dClearSelfpowered(pD);

void* pD;
```

Parameter List

`pD` [in]

driver state info specific to this instance

Description

Clear the device self powered feature.

Return Value

0 on success

EINVAL if pD is NULL

EOPNOTSUPP if feature is not supported

6.24. dVbusSession

Function Declaration

```
uint32_t dVbusSession(pD, isActive);

void* pD;
uint8_t isActive;
```

Parameter List

`isActive` [in]

if = 1 VBUS is ON, if = 0 VBUS is OFF

`pD` [in]

driver state info specific to this instance

Description

Switch VBUS ON or OFF.

Return Value

0 on success

EINVAL if pD is NULL

EOPNOTSUPP if feature is not supported

6.25. dVbusDraw

Function Declaration

```
uint32_t dVbusDraw(pD, mA);

void* pD;
uint8_t mA;
```

Parameter List

`mA` [in]

VBUS maximal current

`pD` [in]

driver state info specific to this instance.

Description

Draw VBUS with given current, real VBUS current should be twice the value given in mA.

Return Value

0 on success

EINVAL if pD is NULL

EOPNOTSUPP if feature is not supported

6.26. dPullUp

Function Declaration

```
uint32_t dPullUp(pD, isOn);

void* pD;
uint8_t isOn;
```

Parameter List

`pD` [in]

driver state info specific to this instance

`isOn` [in]

if 1 device connected, if 0 device is disconnected from host

Description

Disables or enables pullups, which the host may see as disconnect or connect *.

Return Value

0 on success

EINVAL if pD is NULL

EOPNOTSUPP if feature is not supported

6.27. dExecDebug

Function Declaration

```
uint32_t dExecDebug(pD, code, param);

void* pD;
uint32_t code;
uint32_t param;
```

Parameter List

`code` [in]

selected command's code

`param` [in]

parameter associated with command

`pD` [in]

driver state info specific to this instance.

Description

Enables to add any specific extended commands for controlling device.

These command can be invoked from user application.

Return Value

0 on success

EINVAL if pD is NULL

EOPNOTSUPP if feature is not supported

6.28. dGetConfigParams

Function Declaration

```
void dGetConfigParams(pD, configParams);

void* pD;
CH9_ConfigParams* configParams;
```

Parameter List

`configParams` [out]

pointer to CH9_ConfigParams structure

`pD` [in]

driver state info specific to this instance

Description

Returns configuration parameters: U1 exit latency and U2 exit latency Function useful only in Super Speed mode.